

Figure 2: IoT thing server architecture

ZigBee to send data to the gateway server. Thing servers can also have wide area connectivity such as GSM, GPRS, 3G and LTE. On top of this communication infrastructure, IoT uses the MQTT protocol to connect the thing servers with the backend applications.

MQTT messaging protocol

The IoT thing servers may generate various types of data. Different backend applications might be interested in different types of data. The data exchange between the thing servers and the backend applications should happen asynchronously. There should not be any dependency on both of them to be alive at the same time to send and receive data.

To address this kind of a need, a publish/subscribe broker can be used in the architecture. HiveMQ is an open source publish/subscribe broker which uses the MQTT protocol to talk to the IoT thing servers on the one hand and to the backend applications on the other. The MQTT protocol works on the basis of topics. IoT thing servers can register topics with the HiveMQ broker. They will publish sensor data as messages on those topics. The backend servers will subscribe to those messages.

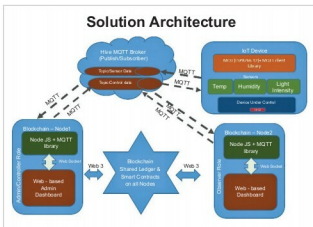


Figure 3: IoT-blockchain PoC architecture

Once an IoT thing server publishes a message on a specific topic, the HiveMQ broker sends that message to all the backend servers which have subscribed to that topic. Similarly, if the backend servers want to send any control information to the IoT thing servers, they can register the control topics with the HiveMQ broker and the IoT thing servers can subscribe to those topics. This will establish a two-way communication between the IoT things and the backend servers using the MQTT protocol and the HiveMQ broker.

PoC

We did a Proof of Concept (PoC) implementation using IoT, blockchain and MQTT to control a transducer. The building blocks of the PoC, and the open source technologies with which they were implemented, are explained below:

- A cloud based HiveMQ server was used as the MQTT publish/subscribe broker. On this broker, the IoT thing server registers a topic on which it publishes the sensor data and the blockchain node registers a topic on which it publishes the control data. In addition, the IoT thing server subscribes to receive the control data while the blockchain node subscribes to receive the sensor data. The HiveMQ broker delivers the published messages to subscribers.
- ESP8266-12 was used as the IoT thing server. This had a temperature, humidity and light sensor. It registered the sensor data topic with the cloud based HiveMQ MQTT broker and published the sensor data on this topic. It subscribed to the control messages from the blockchain node.
- A Node.js Web application, which subscribes to the sensor data topic on the HiveMQ broker for getting the sensor data, was used. It registers the topic on which it publishes the control messages on the HiveMQ broker. Subsequently, it publishes the control messages received from the blockchain. This Web application was connected to an Ethereum blockchain using the web3 JavaScript library. It registers the smart contract on the blockchain. It also posts the sensor data to the smart contract and watches for the asynchronous control messages from the smart contract. The control messages received from the smart contract are published through the HiveMQ broker.
- The smart contract was written using the Solidity language and compiled using a Solc compiler. The compiled smart contract is posted to the Ethereum blockchain using the web3 JavaScript library from the blockchain node.

Logistics use case

Let us again look at the logistics use case mentioned at the beginning of this article and see how the solution can be implemented. The perishable items are packed in a container in which a thing server with the temperature/pressure sensors is fitted. The sensor data is published through the MQTT broker.

Continued on page 81...